

SKILL WEEK-10

1 . To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events

Aim : To register signal handlers, configure signal actions, handle interrupts, manage signal context, verify program stability, and test different signal events.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

volatile sig_atomic_t signal_received = 0;

/* Signal handler */
void signal_handler(int sig)
{
    signal_received = sig;

    if (sig == SIGINT)
    {
        printf("\nSIGINT received - Interrupt handled.\n");
    }
    else if (sig == SIGTERM)
    {
        printf("\nSIGTERM received - Termination request handled.\n");
    }
    else if (sig == SIGUSR1)
    {
        printf("\nSIGUSR1 received - User-defined event handled.\n");
    }
}

int main()
{
    struct sigaction sa;

    printf("Signal Handling Program Started\n");
    printf("Process ID: %d\n", getpid());

    /* Clear structure */
    sa.sa_handler = signal_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    /* Register SIGINT */
    if (sigaction(SIGINT, &sa, NULL) == -1)
    {
        perror("sigaction SIGINT");
        return 1;
    }
}
```

Output :

Terminal-1 :

```
root@keerthana: ~/OSSP
root@keerthana:~# mkdir -p ~/OSSP
cd ~/OSSP
root@keerthana:~/OSSP# pwd
/root/OSSP
root@keerthana:~/OSSP# nano task_signal_actions.c
root@keerthana:~/OSSP# gcc -Wall -Wextra task_signal_actions.c -o task_signal_actions
root@keerthana:~/OSSP# ./task_signal_actions
Signal Handling Program Started
Process ID: 509

Signal handlers registered successfully.
Waiting for signal events...
Press Ctrl+C to test SIGINT.

SIGUSR1 received - User-defined event handled.
Program is still running and stable.

SIGTERM received - Termination request handled.
Program is still running and stable.

SIGINT received - Interrupt handled.
Program is still running and stable.
```

Terminal-2 :

```
root@keerthana: ~/OSSP
root@keerthana:~# cd ~/OSSP
root@keerthana:~/OSSP# ps -ef | grep task_signal_actions
root      509      312  0 14:52 pts/0    00:00:00 ./task_signal_
actions
root      534      515  0 14:52 pts/2    00:00:00 grep --color=a
uto task_signal_actions
root@keerthana:~/OSSP# 509
509: command not found
root@keerthana:~/OSSP# kill -SIGUSR1 509
root@keerthana:~/OSSP# kill -SIGTERM 509
root@keerthana:~/OSSP# kill -SIGINT 509
root@keerthana:~/OSSP# kill -9 509
root@keerthana:~/OSSP# ps -p 509
  PID TTY          TIME CMD
root@keerthana:~/OSSP# |
```

2. Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling

Aim : This task demonstrates how a shell handles Ctrl+C (SIGINT) without terminating the shell itself, while forwarding the signal to the foreground job.

Program :

```
GNU nano 2.9.3
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t foreground_job = -1;

void handle_sigint(int sig)
{
    printf("%d\n", sig);
    if (foreground_job > 0) {
        printf("\nShell: Forwarding SIGINT to foreground job (PID %d)\n",
            foreground_job);
        kill(foreground_job, SIGINT);
    } else {
        printf("\nShell: SIGINT received, but no foreground job is running.\n");
    }
}

int main()
{
    struct sigaction sa;
    pid_t pid;
    int status;

    sa.sa_handler = handle_sigint;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    if (sigaction(SIGINT, &sa, NULL) == -1) {
        perror("sigaction");
        return 1;
    }

    printf("Shell process started\n");
    printf("PID: %d\n", getpid());
    printf("Press Ctrl-C to interrupt the foreground job.\n\n");

    pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }
}
```

```
GNU nano 7.2

pid = fork();
if (pid < 0) {
    perror("fork");
    return 1;
}

if (pid == 0) {
    signal(SIGINT, SIG_DFL);

    printf("Foreground job started.\n");
    printf("Job PID: %d\n", getpid());
    printf("Job State: RUNNING\n");
    printf("Waiting for SIGINT...\n");

    while (1) {
        sleep(1);
    }
}

foreground_job = pid;

printf("Shell: Foreground job PID = %d\n", foreground_job);
printf("Shell: Job State = RUNNING\n");
printf("Shell is waiting for the foreground job...\n\n");

waitpid(foreground_job, &status, 0);

if (WIFSIGNALED(status)) {
    printf("\nShell: Foreground job terminated by signal %d\n",
        WTERMSIG(status));
    printf("Shell: Job State = TERMINATED\n");
}
else if (WIFEXITED(status)) {
    printf("\nShell: Foreground job exited normally.\n");
    printf("Shell: Job State = COMPLETED\n");
}

foreground_job = -1;

printf("Shell process is still running.\n");
printf("Shell process protected successfully.\n");

return 0;
}
```

Output :

```
root@keerthana:~/OSSP# cd ~/OSSP
nano task_sigint_forward.c
root@keerthana:~/OSSP# gcc -Wall -Wextra task_sigint_forward.c -o task_sigint_forward
root@keerthana:~/OSSP# ./task_sigint_forward
Shell Process Started
Shell PID: 557
Press Ctrl+C to interrupt the foreground job.
Shell: Foreground job PID = 558
Shell: Job State = RUNNING
Shell: Waiting for the foreground job.

Foreground job started.
Job PID: 558
Job State: RUNNING
Waiting for SIGINT...
^C
Shell: Forwarding SIGINT to Foreground Job (PID 558)
Shell: Foreground job exited normally
Shell: Job State = COMPLETED

Shell Process is still running.
Shell Process Protected Successfully.
root@keerthana:~/OSSP# |
```

3. Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow

Aim : We can do this as a simple shell job-control simulation. The shell captures `SIGTSTP` (`Ctrl+Z`), sends it to the foreground process, records the job as STOPPED, and later resumes it using `SIGCONT`.

Program :

GNU nano 7.2

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t foreground_job = -1;
int job_state = 0;

/* Handle Ctrl+Z */
void handle_sigstsp(int sig)
{
    (void)sig;

    if (foreground_job > 0) {
        printf("\nShell: SIGTSTP received.\n");
        printf("Shell: Suspending foreground job (PID %d)\n",
            foreground_job);

        kill(foreground_job, SIGSTOP);

        job_state = 1;
        printf("Shell: Job State = STOPPED\n");
        printf("Shell: Job state preserved.\n");
    }
}

int main()
{
    struct sigaction sa;
    pid_t pid;
    int status;

    /* Register SIGTSTP handler */
    sa.sa_handler = handle_sigstsp;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    if (sigaction(SIGTSTP, &sa, NULL) == -1) {
        perror("sigaction");
        return 1;
    }

    printf("Shell Process Started\n");
    printf("Shell PID: %d\n", getpid());

    /* Create foreground job */
```

^G Help
^X Exit

^O Write Out
^R Read File

^W Where Is
^_ Replace

^K Cut
^U Paste

```
/* Create foreground job */
pid = fork();

if (pid < 0) {
    perror("fork");
    return 1;
}

if (pid == 0) {
    /* Child process */
    signal(SIGTSTP, SIG_DFL);

    printf("Foreground job started.\n");
    printf("Job PID: %d\n", getpid());
    printf("Job State: RUNNING\n");
    printf("Job is working...\n");

    while (1) {
        printf("Foreground job is running...\n");
        sleep(2);
    }
}

/* Parent shell */
foreground_job = pid;

printf("\nShell: Foreground job PID = %d\n", foreground_job);
printf("Shell: Job State = RUNNING\n");
printf("Press Ctrl+Z to suspend the job.\n\n");

/* Wait for job status changes */
waitpid(foreground_job, &status, WUNTRACED);

if (WIFSTOPPED(status)) {
    job_state = 1;

    printf("\nShell: Job PID %d has been suspended.\n",
           foreground_job);
    printf("Shell: Job State = STOPPED\n");

    printf("\nPress ENTER to resume the job.\n");
    getchar();

    printf("Shell: Sending SIGCONT to job PID %d\n",
           foreground_job);
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^W Where Is
^_ Replace

^K Cut
^U Paste

GNU nano 7.2

```
foreground_job = pid;

printf("\nShell: Foreground job PID = %d\n", foreground_job);
printf("Shell: Job State = RUNNING\n");
printf("Press Ctrl+Z to suspend the job.\n\n");

/* Wait for job status changes */
waitpid(foreground_job, &status, WUNTRACED);

if (WIFSTOPPED(status)) {
    job_state = 1;

    printf("\nShell: Job PID %d has been suspended.\n",
           foreground_job);
    printf("Shell: Job State = STOPPED\n");

    printf("\nPress ENTER to resume the job.\n");
    getchar();

    printf("Shell: Sending SIGCONT to job PID %d\n",
           foreground_job);

    kill(foreground_job, SIGCONT);

    job_state = 2;

    printf("Shell: Job State = RUNNING\n");
    printf("Shell: Job resumed successfully.\n");

    sleep(5);

    printf("\nShell: Terminating foreground job.\n");
    kill(foreground_job, SIGTERM);

    waitpid(foreground_job, &status, 0);

    printf("Shell: Job State = TERMINATED\n");
}

foreground_job = -1;

printf("Shell: Job removed from foreground.\n");
printf("Shell Process completed.\n");

return 0;
```

^G Help

^O Write Out

^W Where Is

^K Cut

Output :

```
root@keerthana: ~/OSSP  X + v
root@keerthana:~/OSSP# cd ~/OSSP
root@keerthana:~/OSSP# nano task_sigtstp.c
root@keerthana:~/OSSP# gcc -Wall -Wextra task_sigtstp.c -o task_sigtstp
root@keerthana:~/OSSP# ./task_sigtstp
Shell Process Started
Shell PID: 567

Shell: Foreground job PID = 568
Shell: Job State = RUNNING
Press Ctrl+Z to suspend the job.

Foreground job started.
Job PID: 568
Job State: RUNNING
Job is working...
Foreground job is running...
Foreground job is running...
Foreground job is running...
Foreground job is running...
Foreground job is running...
Foreground job is running...
Foreground job is running...
^Z
Shell: SIGTSTP received.
Shell: Suspending foreground job (PID 568)
Shell: Job State = STOPPED
Shell: Job state preserved.
Shell: Job removed from foreground.
Shell Process completed.
root@keerthana:~/OSSP#
```


4. Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior

Aim : This program demonstrates basic job-control process groups. The parent acts as the shell, creates a child process group, transfers terminal control to that group, sends a signal to the group, and then restores terminal control to the shell.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <termios.h>

pid_t shell_pgid;
pid_t child_pgid;

/* Handle Ctrl+C */
void handle_sigint(int sig)
{
    (void)sig;

    printf("\nShell: SIGINT received.\n");

    if (child_pgid > 0) {
        printf("Shell: Forwarding SIGINT to process group %d\n",
            child_pgid);

        kill(-child_pgid, SIGINT);
    }
}

int main()
{
    pid_t pid;
    int status;

    shell_pgid = getpgrp();

    printf("Shell Process Started\n");
    printf("Shell PID      : %d\n", getpid());
    printf("Shell Group ID: %d\n", shell_pgid);

    /* Register SIGINT handler */
    signal(SIGINT, handle_sigint);

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
```

^G Help	^O Write Out	^W Where Is	^K Cut
^X Exit	^R Read File	^_\ Replace	^U Paste

```
/* Parent */
child_pgid = pid;

/* Make sure child is in its own process group */
setpgid(pid, pid);

printf("\nShell: Child created.\n");
printf("Shell: Child PID      = %d\n", pid);
printf("Shell: Child Group ID = %d\n", getpgid(pid));

/*
 * Transfer terminal control to child process group.
 * tcsetpgrp() may fail when the program is not running
 * with a controlling terminal, so report the error.
 */
if (tcsetpgrp(STDIN_FILENO, child_pgid) == -1) {
    perror("tcsetpgrp - transfer terminal");
} else {
    printf("Shell: Terminal control transferred to child group.\n");
}

sleep(3);

printf("\nShell: Sending SIGINT to child process group.\n");
kill(-child_pgid, SIGINT);

sleep(2);

/* Restore terminal control to shell */
if (tcsetpgrp(STDIN_FILENO, shell_pgid) == -1) {
    perror("tcsetpgrp - restore terminal");
} else {
    printf("Shell: Terminal control restored.\n");
}

/* Terminate child */
kill(-child_pgid, SIGTERM);

waitpid(pid, &status, 0);

printf("Shell: Child process terminated.\n");
printf("Shell: Process group management completed.\n");

return 0;
```

```
}
|
```

^G Help	^O Write Out	^W Where Is	^K Cut	^T Exec
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Just

Output :

```
root@keerthana: ~/OSSP
root@keerthana:~/OSSP# pkill -9 -f task_process_groups
root@keerthana:~/OSSP# ps -ef | grep task_process_groups
root      615      592  0 15:02 pts/2    00:00:00 grep --color=auto task_process_groups
root@keerthana:~/OSSP# cd ~/OSSP
nano task_process_groups.c
root@keerthana:~/OSSP# gcc -Wall -Wextra task_process_groups.c -o task_process_groups
root@keerthana:~/OSSP# ./task_process_groups
Shell Process Started
Shell PID: 622
Shell Group ID: 622

Shell: Child created.
Shell: Child PID = 623
Shell: Child Group ID = 623
Shell: Terminal control transferred to child group.

Child Process Started
Child PID : 623
Child Group ID: 623
Child: Waiting for signals..

Shell: Sending SIGINT to child process group.
Shell: Child process terminated.

[1]+  Stopped                  ./task_process_groups
root@keerthana:~/OSSP#
```

